Course: Advanced Algorithm and C++

Week:02

Department: Computer Science

We live under the shared quote

“ Batch 3 - 3 Departments ”

Supported by:

CADT IDT វិទ្យាស្ថានបច្ចេកវិទ្យាឌីជីថល Institute of Digital Technology

Organized by:

NEXT GEN—3 Batch 3, 3 Departments

What We Will Cover in 5 Weeks

Foundation and Memory Management	W1
Sorting Algorithms	W2
Linked List	W3
Stack and Queue	W4
Project	W5

TABLE OF CONTENTS

Learning Objectives

By the end of this session, you will be able to:

- **Explain** what a sorting algorithm does and why sorted data is easier to work with
- **Measure** an algorithm with Big-O notation and state its time and space cost
- **Trace** bubble, insertion and selection sort by hand on a small array
- **Implement** all three sorts in C++ and choose the right one for a given situation

Introduction to Sorting Algorithm

Sorting means rearranging the items of an array or list so they follow an order, usually smallest to largest

Well known sorting algorithms:

- **Bubble Sort -** compare and swap neighbors
- **Insertion Sort -** build a sorted part one item at a time
- **Selection Sort -** repeatedly pick the smallest
- **Merge Sort -** split, sort each half, merge
- **Quick Sort -** partition around a pivot

Why we sort:

- Order turns a messy pile of data into something you can work with
- Can roughly locate what you're searching for instead of going through everything
- Repeats, gaps and extreme values become obvious once things line up
- Sorting is what makes future steps so simple

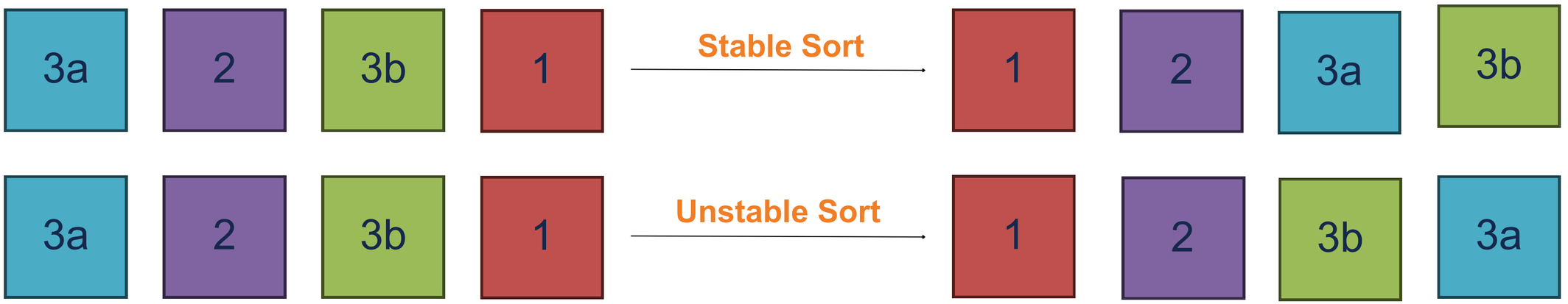
Time and Space Complexity

We compare sorting algorithms by:

- **Time Complexity -** amount of work/comparisons as input grows
- **Space Complexity -** amount of extra memory needed as input grows
- **Stability -** whether items with equal values keep their original input order
- **Writes -** how often the algorithm moves data in memory

Stability, seen on one array

Why consider stability:



Time and Space Complexity

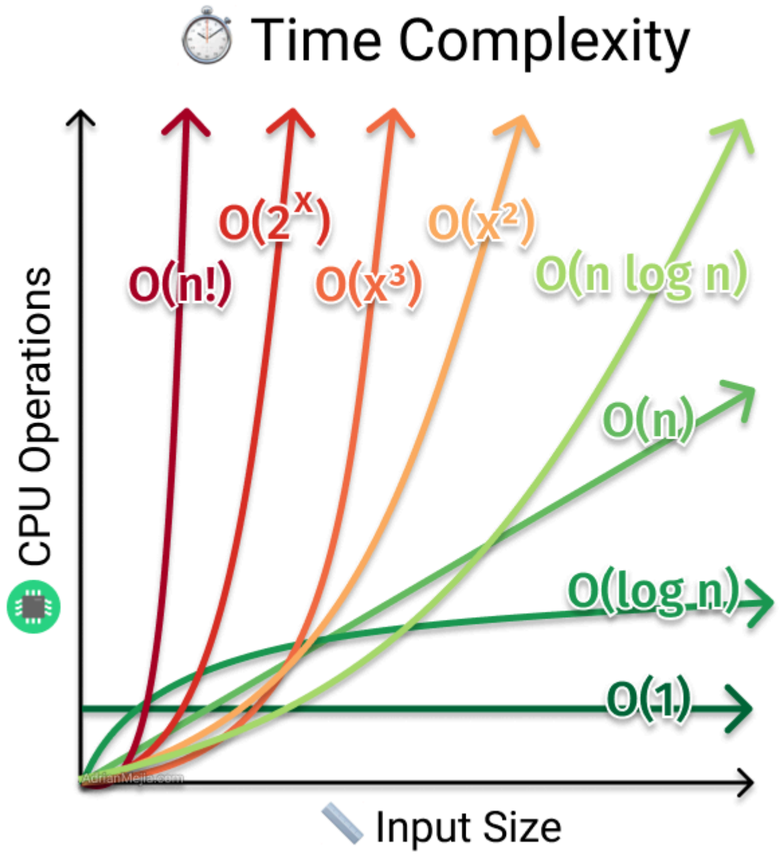
Big-O Notation describes how the work grows as the input size n grows. We ignore constants and keep only the dominant term

Common Time Complexity

- **O(1):** Constant Time
- **O(log n):** Logarithmic
- **O(n):** Linear
- **O(n log n):** Linearithmic
- **O(n²):** Quadratic

Common Space Complexity

- **O(1):** Constant Space
- **O(n):** Storing another copy of input



❗ Space and Time are intertwined, most algorithms have to tradeoff one to optimize the other.

Bubble Sort

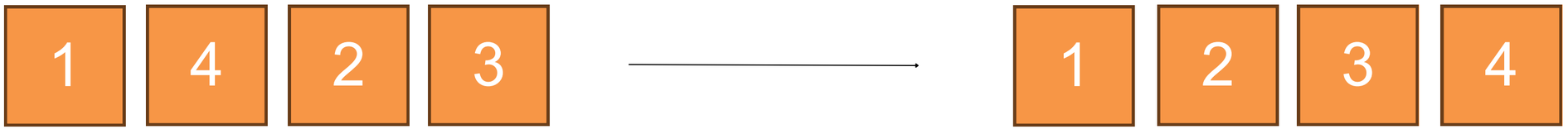
Bubble Sort repeatedly compares two adjacent elements and swaps them if they are in the wrong order. After each pass, the largest remaining element has bubbled up to the end of the array.

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;

        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

Properties

- **Worst Case:** O(n²) - array in reverse order:
- **Best Case:** O(n) - array is already sorted:
- **Space Complexity:** O(1)
- **Stability:** stable
- **Write:** high



Bubble Sort

Let visualize how bubble sort algorithm sorts a list

Insertion Sort

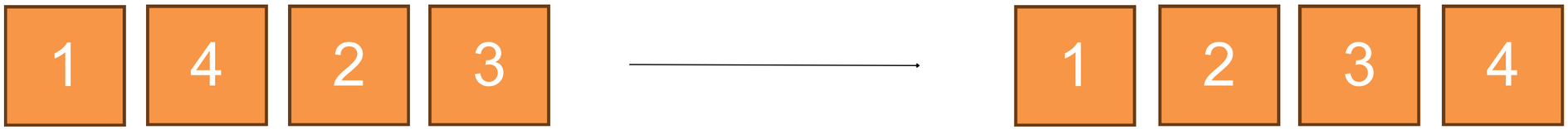
Insertion Sort builds a sorted region at the front of the array. It takes the next element, shifts every larger element one place right, and drops it into the gap. The same way most people sort cards in hand

```
void insertionSort(int arr[] , int n){
    for (int i = 1 ; i < n ; i ++){
        int temp  = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > temp){
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
    }
}
```

Properties

- **Worst Case:** O(n²) - array in reverse order:
- **Best Case:** O(n) - array is already sorted:
- **Space Complexity:** O(1)
- **Stability:** stable
- **Write:** high



Insertion Sort

Let visualize how insertion sort algorithm sorts a list

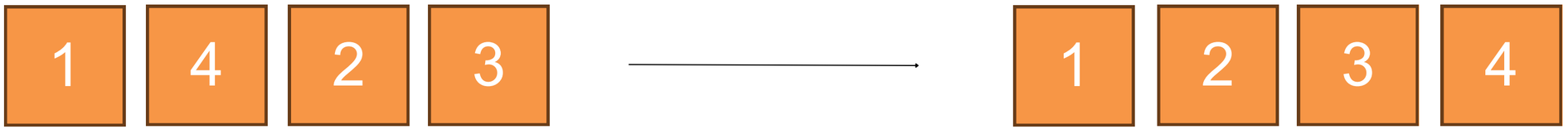
Selection Sort

Selection Sort splits the array into a sorted and an unsorted part. Each pass scans the unsorted part for the minimum and swaps it into the boundary position, growing the sorted part by one

```
void selectionSort(int arr[] , int n){
    for (int i = 0 ; i < n ; i++){
        int min = i;
        for (int j = i + 1 ; j < n ; j++){
            if (arr[min] > arr[j]){
                min = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
}
```

Properties

- **Worst Case:** O(n²) Array in reverse order:
- **Best Case:** O(n²) Array is already sorted:
- **Space Complexity:** O(1)
- **Stability:** unstable
- **Write:** low



Selection Sort

Let visualize how selection sort algorithm sorts a list

Choosing the Right Sort

All three are $O(n^2)$ and all sort in place, so the choice comes down to the shape of your data

Algorithm	Best	Average/worst	Space	Stable	Writes	Pick it when
Bubble	O(n)	$O(n^2)$	O(1)	Yes	High	Teaching the idea, tiny arrays
Insertion	O(n)	$O(n^2)$	O(1)	Yes	High	Small or nearly sorted data
Selection	$O(n^2)$	$O(n^2)$	O(1)	No	Low	Writing to memory is expensive

Lab Exercise

1. **Bubble Sort**

Write **void bubbleSort(int arr[] , int n)** using two plain loops, no early exit. Fill an array with **{ 5 , 1 , 4 , 2 , 8 }** in main, print it before and after sorting

2. **Early Exit**

Add a **swapped** flag so the sort stops once a pass makes no swaps. Print a message when it exits early, then run it on **{ 1 , 2 , 4 , 5 , 8 }** to prove it stops after one pass

3. **Insertion Sort**

Write **void insertionSort(int arr[] , int n).** Print the whole array at the end of every outer loop so you can watch the sorted region grow.

4. **Selection Sort**

Write **void selectionSort(int arr[] , int n)**. Keep the index of the minimum, not the value, and do exactly one swap per pass.

Lab Exercise

1. **Count the Work**

Add a **comparison counter** and a **swap counter** to **all three sorts**. Run them on the same array and print the totals. Which one moves data the least?

2. **Sort Your Structs**

Take the **Book struct** from week 1 and write a function that sorts an array of 5 books by **year** using any of the three algorithms.

3. **Descending Order**

Change one comparison in each sort so the array comes out largest first. Confirm it still works on an array that contains **duplicates**.

4. **Prove Stability**

Sort **{"Alice", 3}, {"Bob", 1}, {"Chan", 3}, {"Dara", 2}** by score with insertion sort, then with selection sort. Print both and explain why Alice and Chan swap places

References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

[2] Deitel, P., & Deitel, H. (2017). C++ how to program (10th ed.). Pearson.

[3] std::sort. (n.d.). cppreference.com. https://en.cppreference.com/w/cpp/algorithm/sort

[4] Sorting algorithms. (n.d.). LearnCpp.com. https://www.learncpp.com/cpp-tutorial/sorting-an-array-using-selection-sort/

Thank You!